



Holy Angel University
Angeles City



Name: Carosus, Cean A.

Date: September 18, 2026

Section: CS - 302

Course: Computer Science

Project Proposal Version 2

1. App Name

→ AGOS

2. The problem, in one sentence

Citizens who encounter illegally parked vehicles have an outlet to vent, public shaming pages on Facebook, but no structured way to document the incident and get it in front of the people who can actually act on it.

3. Who is this for?

- **Citizen reporters:** commuters, motorists, cyclists, and pedestrians in Angeles City who currently deal with illegally parked vehicles by posting to pages like **ParkSerye**, a public anonymous Facebook page where people share photos of offending vehicles. The post gets comments and the vehicle gets called out, but then nothing happens. No record of whether it was moved, no one officially notified, no way to check back later.
- **Authority reviewers** :Local Government Units (LGU), who currently have no single place to see what's being reported across the city in real time, only whatever surfaces on social media or reaches them by phone.

4. Core Features (MVP)

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest Estimate
1	Login with Seeded Accounts	Kept & Expanded	TextField, FirebaseAuth.instance.signInWithEmailAndPassword, simple account selector buttons for demo convenience, StreamBuilder or Provider for state, Navigator.pushReplacement based on user role (citizen vs authority).	1 Hour
2	Submit Report (with Location Capture)	Combined(Keep)	Form, TextField, image_picker plugin (ImagePicker().pickImage) for photo, geolocator plugin (Geolocator.getCurrentPosition) fallback to manual location text entry,	10 Hours

			ElevatedButton, Firebase Storage upload, Firestore record creation.	
3	Citizen Dashboard & Modal Report Details	Combined (Keep)	StreamBuilder<QuerySnapshot> reading Firestore filtered by reporterId, ListView.builder, Card, Chip for status. Tapping a report triggers showDialog() displaying a centered modal dialog with photo, location, date, ID, and status badge over a darkened backdrop scrim.	2 Hours
4	Track Report Status & Authority Review	Added (Authority Review) & Combined with Track Status	StreamBuilder<QuerySnapshot> reading all active reports, DropdownButton / SegmentedButton filter by status, ListTile, detail inspection, and status updates via FirebaseFirestore.instance.collection('reports').doc(id).update({'status': newStatus})	8 Hours

5. Stretch Goals

- Real-time push notifications (using Firebase Cloud Messaging) to notify citizens when an authority updates their report status.
- User registration / Sign-up flow for non-seeded accounts.
- AI-powered image analysis to detect illegally parked vehicles automatically.
- Integration with LGU dashboards for report management and monitoring.
- CCTV integration for automated road obstruction detection.

6. How my app saves data

A. If two different people install my app, should they see the same data?

Yes. Citizens submit reports that authority reviewers must inspect and update. An authority user on one device needs to see live data submitted by a citizen on another device, and the citizen must see when the authority marks their report as "Resolved" or "Rejected."

B. Roughly how many records does my app hold in a realistic week of use?

Around 30–50 reports total during seeded testing and demo execution, across 3 pre-configured seeded accounts (2 citizen accounts and 1 authority account).

C. My choice

Firebase (Cloud Firestore for database, Firebase Authentication for seeded accounts, and Firebase Storage for photo evidence).

D. Why this one and not the others, in two or three sentences. Name the tradeoff you accepted.

shared_preferences was considered in my initial V2 draft, but it isolates data to a single local device. Because AGOS requires a two-role system where an authority updates a report that a citizen submitted, the data must be shared, synchronized, and persisted in a real cloud database. The tradeoff accepted is a higher setup overhead (configuring Firebase projects, security rules, environment variables, and handling asynchronous network states) and reliance on an active internet connection during testing.

E. What I save, concretely: which class, which fields, under what key or in which table or collection.

- Collection users: uid (doc ID), email, fullName, role ("citizen" or "authority"). Pre-seeded with 2 citizen accounts and 1 authority account.
- Collection reports: id (doc ID), reporterId, photoUrl, description, latitude (double), longitude (double), locationName (String), timestamp (DateTime), status ("Pending", "Resolved", or "Rejected").

F. Have I tried it yet? If you did the one-hour spike from page 6, say what happened. If not, say when you will.

In my previous iteration, I spiked local JSON encoding with shared_preferences. Moving to Firebase is a newly proposed architectural shift for this revised proposal; I have not yet wired the live Firebase SDK into the main Flutter build, which is scheduled for the upcoming development cycle

7. One thing I want to add that the course did not teach

A. Package

geolocator combined with image_picker.

B. Does it run where I develop

image_picker and geolocator both run on Android and iOS. On the web (flutter run -d web-server), location permissions work via browser Geolocation API, while image_picker falls back to standard file-picker dialogs.

C. Web fallback

If GPS access is denied or fails during web development, the location capture form gracefully falls back to a manual text input field (`TextFormField`) so report submission can be tested end-to-end without crashing.

D. Core or stretch:

Core. Capturing real-time location data is fundamental to reporting an illegal parking incident accurately without forcing the user to manually type out full address coordinates.

8. How my project runs when someone else opens it

A. I am keeping the device_preview wrapper

Yes. It allows multi-screen testing across different mobile viewport sizes on a single simulator, ensuring dialogs and report lists fit well

B. My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable

Not yet, but planned. Hardware dependencies (camera and GPS) degrade to browser file-picker dialogs and manual text input fallbacks.

C. Anything that needs real hardware degrades to sample data instead of crashing

Yes. If GPS is unavailable, manual text entry activates; if camera hardware is unavailable, `image_picker` opens local file storage.

D. My project will live in a public repository in my own GitHub account. Does anything I plan to build need an API key, a password, or real personal data?

Yes. Firebase integration requires configuration options (`google-services.json`, `firebase_options.dart`). The 3 seeded accounts will use dummy credentials (e.g., `citizen1@agos.app`, `citizen2@agos.app`, `authority1@agos.app`). API keys and configuration values will be loaded via environment configs (`.env` file added to `.gitignore`, with a `.env.example` committed to the repository).

9. Data the App Needs to Remember

Thing	Fields	Where it is saved
-------	--------	-------------------

User	uid, email, fullName, role ("citizen" or "authority")	Cloud Firestore (collection: users) & Firebase Auth (Pre-seeded for 2 citizens, 1 authority)
Report	id, reporterId, photoUrl, description, latitude, longitude, locationName, timestamp, status	Cloud Firestore (collection: reports) & Firebase Storage (photos)

10. Screens You'll Need

Screen	Why It Must Exist	Data Owned / Shown	Real Content to Gather
Login Screen (Shared Landing)	Allows the user/tester to log in directly using one of the pre-configured seeded accounts (Citizen 1, Citizen 2, or Authority 1) without going through a sign-up process, routing them to the correct role interface.	Credentials for 2 citizen accounts and 1 authority account.	Username and password for each seeded account
Citizen Home Dashboard & History Screen (with Modal Report Details)	Serves as the primary hub for citizen reporters. Lists all submitted reports with status badges (Pending, Resolved, etc.) and provides access to submit new reports. Clicking any report item triggers a centered modal dialog (showDialog) over a darkened backdrop scrim to inspect full photo evidence, violation description, location text, date/time, report ID, and status badge without leaving the dashboard view.	List of Report items where <code>reporterId == currentUserId</code> . Selected Report object for the active modal dialog.	Total/Pending summary cards, <code>ListView.builder</code> report cards, Floating Action Button, and centered Dialog modal overlay with a "Close Details" button,
Submit Report	Allows citizens to capture/upload a photo, auto-detect location (or manually enter text), enter a description, and upload to Firebase.	New Report instance (photo file, location coordinates/text, description).	Image upload preview box with icon, auto-detect location button and text field, multiline description field, and "Submit Report" button.

Authority Review Dashboard	Dedicated view for local authorities to monitor incoming municipal parking reports across Angeles City, filter by status (Pending, Resolved, Rejected), and select reports for inspection.	All Report records across all users in Cloud Firestore.	Jurisdiction header, total/pending/resolved metric cards, [All Pending Resolved Rejected] status filter bar, and incoming report list cards.
Authority Report Action & Resolution Screen	Dedicated screen for enforcement officers to inspect photo evidence, view exact map location coordinates, write resolution notes, and update report statuses in real time.	Selected Report document (reporter name, full photo, GPS coordinates, description, and status).	Full-width vehicle photo, reporter details, interactive status selector (Pending, Resolved, Rejected), authority remarks text box, and "✓ Update & Save Status" button

11. Risks, revised

A. The risk I named last time

“The biggest risk is reliably implementing photo uploads and GPS location capture while ensuring that report information is correctly stored and retrieved from the application's data storage. If these features are not implemented correctly, users may submit incomplete reports or be unable to view their previously submitted reports and their current statuses. “

→ Is it still a risk? How has it evolved?

It is still a core risk, but its specifics have evolved. In the preliminary proposal, I had not fully defined how media and report data would be stored or synchronized. With the decision to move to Firebase (Cloud Firestore, Authentication, and Firebase Storage), the risk shifts from vague storage failures to concrete cloud implementation challenges:

- Ensuring local photos picked via image_picker upload reliably to Firebase Storage and return valid network URLs.
- Making sure geolocator GPS coordinates attach cleanly to the Firestore document.
- Ensuring that when an Authority updates a report status in Firestore, active real-time listeners (StreamBuilder) immediately reflect that change on the Citizen's UI without requiring a manual refresh.

B. A new risk I did not see before

Role Authorization & Pre-Seeded Account Synchronization: Restoring report status tracking and creating a two-role workflow (Citizen vs. Authority) introduces security and state risks. If Firestore rules or state checks are misconfigured, a Citizen could accidentally modify a report's status, or an Authority might be unable to write updates to another user's submitted report document..

C. First step to reduce each, and when

- **Image & GPS Cloud Upload Risk:** Write a standalone test script that picks an image, fetches current GPS coordinates, uploads the image file to Firebase Storage, writes the document to Firestore, and renders the result back in an `Image.network` widget by **September 21, 2026**.
- **Role Authorization Risk:** Write basic Firestore security rules enforcing read/write permissions per role and test seamless role switching and real-time state updates between the 3 seeded test accounts (Citizen 1, Citizen 2, Authority 1) on separate devices/simulators by **September 23, 2026**.

12. What changed, and why

Reflection on the Prelim Proposal vs. Revised Architecture

When I re-evaluated my preliminary proposal, I realized it was full of critical logic holes because I had planned it on a whim rather than thoroughly thinking through how a real software application works.

For instance, my prelim proposal completely omitted Login as a core feature, even though an app cannot attribute reports to a specific user or protect administrative actions without authentication. More importantly, the original plan never answered the fundamental question: *Where do the submitted reports actually go, and who processes them?* A citizen submitting an illegal parking report into a void where no authority can view or resolve it completely defeats the purpose of the app.

Additionally, several features in the prelim MVP table were artificially separated (such as making "Submit Report" and "Location Capture" two distinct core rows, or separating "Home Dashboard" from "View Report History"), making the project plan fragmented.

In this revision, I deliberately corrected these bad decisions:

- I restored the full end-to-end loop by explicitly introducing the Authority / Reviewer role Combined with Tracking the Report Status to the core MVP.
- I added Login into the core MVP, streamlining it with 3 pre-seeded accounts (2 citizens, 1 authority) so testing is seamless without needing a full registration/sign-up build.
- I consolidated fragmented features into logical, unified workflows (e.g., Dashboard, Report History and Report Details into just one screen instead of separated ones).

Section	Prelim Said	Now Says	Why it Changed
The problem	"Citizens who encounter illegally parked vehicles often	"Citizens...have an outlet to vent, public shaming pages on	The original was accurate but generic. Naming a real existing

	lack a convenient and centralized way to report them to the appropriate authorities"	Facebook, but no structured way to document the incident and get it in front of the people who can actually act on it"	alternative (Facebook shaming pages) and stating the actual gap precisely makes the problem concrete instead of abstract.
Who is this for	One group: "commuters, motorists, cyclists, and pedestrians who encounter illegally parked vehicles"	Two groups: Citizen reporters (anchored to a real alternative, ParkSerye) and Authority reviewers.	Prelim scored 6/8 on Purpose & Audience with feedback naming this exact gap: the audience stayed broad, and naming one commuter's specific reporting habit would sharpen it. Anchoring to <i>ParkSerye</i> and splitting out the two distinct user types does that directly.
Core Feature 1	(Not listed as a core MVP feature row)	Login with Seeded Accounts (Added)	Added because authentication is required to route users to the correct interface (citizen vs authority). Pre-seeding 3 accounts eliminates the need to build complex sign-up screens while preserving demo convenience.
Core Feature 2	Listed as two separate MVP rows: Submit Report (Row 1) and Location Capture (Row 4).	Submit Report & Location Capture (Combined in Row 2)	Submit Report and Location Capture belong together. Combining them into a single row reflects the actual user action: taking a photo, typing a description, and fetching GPS coordinates on one screen.
Core Feature 3	View Report History (Row 2 in prelim MVP table).	Citizen Dashboard & Modal Report Details (Expanded in Row 3)	In the prelim, View Report History was listed as a basic list page. I expanded this into a full Citizen Dashboard that shows report metrics and lists previous reports.

			Tapping any report opens a centered modal dialog (showDialog) over a darkened backdrop scrim to inspect details without leaving the screen.
Core Feature 4	Track Report Status ("The user selects one of their submitted reports... displays report's current status").	Track Report Status & Authority Review (Expanded in Row 4)	In the prelim, report status was static because no authority existed to change it. Expanding this feature allows authority users to inspect live reports and update their statuses
Screens	5 screens: Login, Home Dashboard, Submit Report, Report History, Report Details	5 Primary Screens + 1 Modal Overlay: Login Screen (Seeded Accounts), Citizen Home Dashboard (with Modal Details), Submit Report Screen, Authority Review Dashboard, and Authority Report Action & Resolution Screen.	1. Merging Citizen Dashboard, History & Details: In the preliminary proposal, Home Dashboard and Report History were separate routes even though both simply listed the user's submitted reports. Merging them into a single Citizen Dashboard eliminates redundant navigation. Additionally, instead of navigating to a dedicated Report Details screen, clicking a report now opens an in-place centered modal dialog (showDialog) over a darkened backdrop scrim. This preserves visual context, reduces route complexity, and avoids managing back-stack arguments. 2. Adding Authority Dashboard & Report Action Screen: To resolve the critical architectural hole in the preliminary

			proposal—where reports were submitted into a void with no authority to process them—two dedicated authority screens were added. The Authority Review Dashboard gives enforcement officers a city-wide feed of incoming reports filtered by status (Pending, Resolved, Rejected), and the Authority Report Action Screen provides a dedicated inspection view to examine photo evidence, verify map coordinates, write resolution notes, and update report statuses in real time.
--	--	--	---